Computers & Security 139 (2024) 103706

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

# BenchMFC: A benchmark dataset for trustworthy malware family classification under concept drift

Yongkang Jiang [a], Gaolei Li [a,*], Shenghong Li [a,*], Ying Guo [b]

[a] Shanghai Jiao Tong University, Shanghai, China
[b] North China University of Technology, Beijing, China

ARTICLE INFO

*Keywords:*
Malware family classification
Machine learning
Concept drift
Benchmark dataset

ABSTRACT

Concept drift poses a critical challenge in deploying machine learning models to mitigate practical malware threats. It refers to the phenomenon that the distribution of test data changes over time, gradually deviating from the original training data and degrading model performance. A promising direction for addressing concept drift is to detect drift samples and then retrain the model. However, this field currently lacks a unified, well-curated, and comprehensive benchmark, which often leads to unfair comparisons and inconclusive outcomes. To improve the evaluation and advance further, this paper presents a new Benchmark dataset for trustworthy Malware Family Classification (BenchMFC), which includes 223 K samples of 526 families that evolve over years. BenchMFC provides clear family, packer, and timestamp tags for each sample, it thus can support research on three types of malware concept drift: 1) unseen families, 2) packed families, and 3) evolved families. To collect unpacked family samples from large-scale candidates, we introduce a novel crowdsourcing malware annotation pipeline, which unifies packing detection and family annotation as a consensus inference problem to prevent costly packing detection. Moreover, we provide two case studies to illustrate the application of BenchMFC in 1) concept drift detection and 2) model retraining. The first case demonstrates the impact of three types of malware concept drift and compares nine notable concept drift detectors. The results show that existing detectors have their own advantages in dealing with different types of malware concept drift, and there is still room for improvement in malware concept drift detection. The second case explores how static feature-based machine learning operates on packed samples when retraining a model. The experiments illustrate that packers do preserve some kind of signals that appear to be "effective" for machine learning models, but the robustness of these signals requires further research. BenchMFC has been released to the community at https://github.com/crowdma/benchmfc.

## 1. Introduction

Machine learning has shown encouraging results for malware family classification (Ucci et al., 2019; Qiu et al., 2020), making it the most likely avenue to handle an ever-increasing number of malicious samples. However, such great success rests on one central assumption that test and training data are drawn identically and independently from the same distribution (i.i.d.) (Bendale and Boult, 2015).

In a real security setting, this assumption does not hold. The distribution of test data tends to change over time, gradually deviating from the original training data and degrading model performance. This phenomenon is known as concept drift (Jordaney et al., 2017; Yang et al., 2021d). As illustrated in Fig. 1, for malware family classification, drift samples can be caused by ***T1: label (semantic) shift*** (Yang et al., 2021a), a change in the label space distribution $P(Y)$ (*e.g., drift samples are drawn from unseen families*), and ***T2: covariate shift*** (Yang et al., 2023), a change in the input space distribution $P(X)$ (*e.g., drift samples from known but packed or evolved families*). For profit, malware authors are keen to exploit novel vulnerabilities to develop new families. These *unseen families*, which are not visible during the training phase, lead to the label shift. In addition, to evade anti-malware systems, malware authors often improve attack techniques of know families to obtain evolved variants, or they just use packing techniques to compress and encrypt samples to present different static features. These *evolved families* and *packed families* introduce the covariate shift. Each type of drift samples has its own characteristics, and there is currently no silver bullet detector that performs well on all drift types (Dib et al., 2022; Yang et al., 2022). Subdividing drift samples helps to study their respective

* Corresponding authors.
*E-mail addresses:* jiangyongkang@sjtu.edu.cn (Y. Jiang), gaolei_li@sjtu.edu.cn (G. Li), shli@sjtu.edu.cn (S. Li), guoying@ncut.edu.cn (Y. Guo).

https://doi.org/10.1016/j.cose.2024.103706
Received 4 August 2023; Received in revised form 18 November 2023; Accepted 4 January 2024
Available online 9 January 2024
0167-4048/© 2024 Elsevier Ltd. All rights reserved.

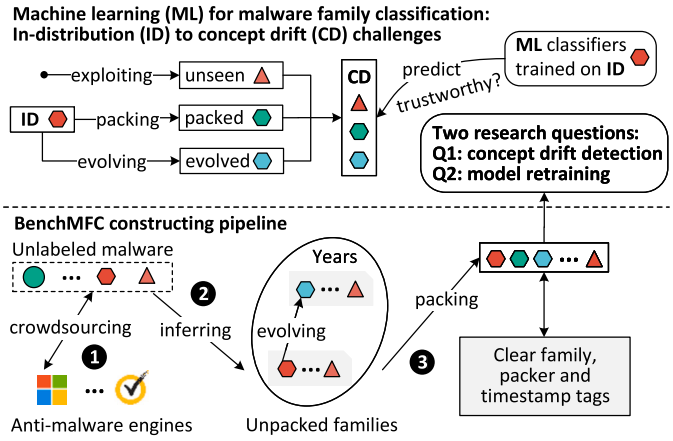



**Fig. 1.** Illustration of BenchMFC in supporting research on trustworthy malware family classification under concept drift. *BenchMFC provides diverse, unpacked, and time-evolving family samples to help simulate unseen, packed and evolved families.* (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

characteristics and develop corresponding mitigation strategies (Yang et al., 2021d; Aghakhani et al., 2020; Zhang et al., 2020; Barbero et al., 2022).

One promising direction to mitigate concept drift is first to detect and isolate potentially misclassified samples until they can be expertly analyzed, and then retrain the model with the new labeled samples (Chen et al., 2023). The former research question is referred to as ***Q1: concept drift detection*** and the latter is called ***Q2: model retraining***. Recent efforts such as Transcend (Jordaney et al., 2017; Barbero et al., 2022) and Cade (Yang et al., 2021d) have conducted preliminary studies on ***Q1***. However, due to the challenge of malware annotation and the fact that few efforts have opened up their datasets, this field currently lacks a uniform, well-curated, and comprehensive benchmark, making it difficult to conduct a fair comparison of existing concept drift detectors and keep track of the latest advances. As for ***Q2: model retraining***, malware researchers have often assumed that packing would hinder machine learning to build effective classifiers on static malware features (Aghakhani et al., 2020; Moser et al., 2007a; Fleshman et al., 2018; Qiang et al., 2022). However, recent classifiers always tell success stories, leading many experts to wonder whether the test benchmarks are biased (few datasets provide clear family and packer tags) or whether the classifiers just learn spurious correlations.

To address the above dilemmas, a new benchmark is needed, for which we can determine whether each sample is 1) packed or unpacked, 2) which family it belongs to, and 3) when it first appeared. Building such a fine-grained dataset faces several challenges, the trickiest of which is how to obtain quality-controlled unpacked family samples.

***Inferring unpacked family samples.*** If there are few candidate samples, the latest hybrid approach introduced by Packware (Aghakhani et al., 2020) would be the best choice to detect packing and obtain unpack samples, because its facts are mainly based on the runtime behavior of the samples, which gives us a high level of confidence in unpacked samples. But our goal is to make the families in the benchmark as diverse as possible, thus we collect 9.4 million candidate malicious samples from ViusShare (Forensics, 2011). Still consider Packware's approach, assuming that each sample is run for 30 seconds to monitor its behavior, it would take over 8 years of analysis, which is not desirable.

Therefore, we introduce a novel crowdsourcing malware annotation pipeline (Section 3.2), which unifies packing detection and family annotation as a consensus inference problem among anti-malware engines to prevent costly packing detection while outputting unpacked family samples. Specifically, the pipeline includes family consensus voting and static packer identification. Take sample $x_i$ as an example, we first upload it to VirusTotal (VirusTotal, 2013) to collect malware labels reported by 70+ anti-malware engines. Then we extract *family tags* from each malware label and vote to obtain its *family consensus* among anti-malware engines. If there are $C_i$ engines that label the sample as family $k$, and $C_i$ is the highest vote, then the family consensus for the sample is noted as $C_i$. In addition, the sample needs to be analyzed by several static packer identification tools. The sample is treated as an unpacked sample of family $k$ if and only if no packers are recognized and its family consensus $C_i \geq 4$. Certainly, our pipeline does not perform generic packing detection, but evaluations on the Packware dataset (Aghakhani et al., 2020) show that it achieves 97.98% recall and 98.29% precision in collecting unpacked family samples (Section 4). After inferring, we receive 4.1 million (43.6% of the collected) unpacked samples of 3,049 families. We filter out small families with $\leq 50$ samples, and downsample large families to 1000 samples. Finally, BenchMFC includes 223 K unpacked samples of 526 malware families, of which about 25% of families have 1000 samples, 25% of families have 50-100 samples, and the remaining 50% of families have an average of 342 samples.

***First seen.*** Another tricky issue is getting the time when the sample first appeared. Accurately obtaining the time of emergence of the wild samples seems to be impossible. A compromise is to use the time when the sample was first seen as an estimate of its time of appearance. BenchMFC takes the minimum of the time the sample was collected and the time VirusTotal (VirusTotal, 2013) recorded the sample as the *first seen* of the sample.

BenchMFC provides clear family, packer, and timestamp tags for each sample, it thus can support research on three types of malware concept drift: 1) unseen families, 2) packed families, and 3) evolved families. To illustrate the application of BenchMFC on concept drift detection and model retraining, two case studies are conducted. This paper focuses on static feature-based machine learning for malware family classification, we extract two groups of widely used static features: EMBER (Anderson and Roth, 2018) and Byte (Raff et al., 2018a). Although we train classifiers MalConv (Raff et al., 2018a), GBDT (Anderson and Roth, 2018), and MLP (Hornik et al., 1989), we mainly discuss the results of the MLP, as we observed similar findings for other classifiers and the MLP classifier is a typical machine learning baseline.

The first case study is about malware concept drift detection. We consider nine notable concept drift detectors, such as ViM (Wang et al., 2022), KL-Matching (Hendrycks et al., 2022), and Mahalanobis (Lee et al., 2018) from the computer vision community, and Cade (Yang et al., 2021d), Transcend (Barbero et al., 2022) from the security community. The experiments demonstrate (Section 5.2) that different types of malware concept drift can significantly degrade the performance of machine learning classifiers, and that existing detectors have their own advantages in dealing with different types of malware concept drift, malware concept drift detection still has room for improvement.

The second case study explores how machine learning operates on packed malware samples when retraining a model. The experiments show (Section 5.3) that packers do preserve some kind of static signals that appear to be "effective" for machine learning classifiers. Due to the lack of interpretability, the robustness of these signals requires further research.

In summary, this paper makes the following contributions:

- We release a new dataset (BenchMFC) for benchmarking trustworthy malware family classification under concept drift, which includes 223 K unpacked samples of 526 families that evolve over years. BenchMFC can support research on concept drift detection and model retraining.
- To collect unpacked family samples from large-scale candidates, we introduce a novel crowdsourcing malware annotation pipeline, which unifies packing detection and family annotation as a consensus inference problem to prevent costly packing detection.
- We provide two case studies to illustrate the application of BenchMFC in: concept drift detection and model retraining. The

**Table 1**
Related PE malware datasets. ○ = "not available", ◐ = "partially available", ● = "available".

| Dataset | Packer | Family | Evolution | #Family [1] | #Sample | First seen |
|---|---|---|---|---|---|---|
| MalImg (Nataraj et al., 2011) | ○ | ● | ○ | 25 | 9,339 | Before 2011 |
| BIG-2015 (Ronen and Radu, 2018) | ○ | ● | ○ | 9 | 10,868 | Before 2015 |
| EMBER (Anderson and Roth, 2018) | ○ | ◐ | ○ | - | 800,000 | 1/2017-12/2018 |
| Packware (Aghakhani et al., 2020) | ◐ | ○ | ○ | - | 232,415 | 1/2017-03/2018 |
| SOREL-20M (Harang and Rudd, 2020) | ○ | ○ | ○ | - | 9,962,820 | 1/2017-04/2019 |
| BODMAS (Yang et al., 2021c) | ○ | ● | ◐ | 102 | 54,669 | 8/2019-09/2020 |
| **BenchMFC (Our)** | ● | ● | ● | **526** | 223,650 | **01/2012-12/2022** |

[1] Only families ≥ 40 samples are counted.

experiments show missing elements in existing research and discuss how the benchmark can support future research.

The rest of the paper is organized as follows: Section 2 describes the background and related work, and Section 4 outlines the properties of BenchMFC. In Section 5, we conduct two case studies to illustrate the application of BenchMFC. Our goal is to demonstrate that *BenchMFC can serve as a central research resource for trustworthy malware family classification under concept drift*. In addition, constructing such a fine-grained benchmark can no longer rely on traditional annotation methods, Section 3 presents how BenchMFC is constructed.

## 2. Background and related work

In this section, we describe the background of malware family classification, concept drift, and illustrate why we introduce BenchMFC.

### 2.1. Machine learning for malware family classification

Malware classification typically refers to one of two related tasks: 1) malware detection (determining whether a given sample is malicious or benign), and 2) malware family classification (differentiating between two or more known malware families). Classifying malware into a known family provides valuable clues about the behavior they may perform and aids in attribution (Oz et al., 2022). Over the past few years, the need for techniques that generalize to unknown malware samples while reducing costly human experts in the loop, has led to the development of machine learning classifiers based on static and dynamic malware features (Ucci et al., 2019; Or-Meir et al., 2019; Raff et al., 2018a; Li et al., 2021).

***Static vs. Dynamic.*** Static analysis and dynamic analysis have their pros and cons (Damodaran et al., 2015). Dynamic analysis extracts features by executing a sample in a controlled environment, it thus provides a clear picture of the behavior of a sample. But it suffers some limitations. For example, dynamic analysis can be costly, especially when dealing with complex or sophisticated malware (Agarwal and Raj, 2018). Besides, advanced malware may employ evasion techniques to detect virtual environments and alter their behavior accordingly to avoid detection (Raffetseder et al., 2007; Lindorfer et al., 2011), which can also limit the effectiveness of dynamic analysis. To achieve fast classification, features derived only from static analysis (Nataraj et al., 2011; Raff et al., 2018b; Yan et al., 2019) (*i.e.,* not executing a sample) are widely explored in recent years to build machine learning models. However, static analysis has known issues when dealing with obfuscated, packed, and adversarial samples (Aghakhani et al., 2020; Moser et al., 2007b; Demetrio et al., 2021; Tran et al., 2022).

### 2.2. Concept drift dilemma

Concept drift poses a critical challenge in deploying machine learning models based on static and dynamic features to address practical malware threats. The distribution of test data tends to change over time, gradually deviating from the original training and degrading model performance. There are generally two types of distribution shifts: ***T1: label (semantic) shift*** and ***T2: covariate shift***. Formally, let $P(X)$ and $P(Y)$ be the distribution of input space and label space of the training data, respectively. Consider test distribution $P'(X)$ and $P'(Y)$, label shift occurs when $P'(Y) \neq P(Y)$ and $P'(X) \neq P'(X)$ (shift in $P(Y)$ naturally triggers shift in $P(X)$), while covariate shift occurs when $P'(X) \neq P(X)$ and $P'(Y) = P(Y)$. In this paper, the above two distribution shifts are further subdivided into three types of drift samples: unseen families (***T1***), packed families (***T2***), and evolved families (***T2***). Because each type of drift samples has its own characteristics, and there is currently no silver bullet that performs well on all drift types (Dib et al., 2022; Yang et al., 2022). This subdividing helps to study their respective characteristics and develop corresponding mitigation strategies (Yang et al., 2021d; Aghakhani et al., 2020; Zhang et al., 2020; Barbero et al., 2022). Specifically, unseen and evolved families involve changes in both dynamic and static characteristics, while packed families only involve changes in static. Thus, classifiers based on dynamic features suffer from unseen and evolved families, while classifiers based on static features suffer from all three types.

The security community has been studying concept drift for several years (Wadkar et al., 2020; Paul and Stamp, 2021; Tupadha and Stamp, 2022). Notable drift detectors include Transcend (Jordaney et al., 2017; Barbero et al., 2022), APIGraph (Zhang et al., 2020), and Cade (Yang et al., 2021d). Transcend is based on conformal prediction theory (Shafer and Vovk, 2008) to reject drift samples. APIGraph is a graph-based approach that detects drift samples by comparing the similarity of API call graphs. They both focus on the malware detection task and conduct evaluations with samples from different timelines. That is, the specific proportion of drift types is unknown. Cade builds on supervised contrastive learning and a distance-based method to detect drift samples, and its evaluations show that Cade outperforms Transcend on unseen families. But the latest study (Dib et al., 2022) shows that Cade is not that effective on evolved families. Concept drift detection (or generalized out-of-distribution detection (Yang et al., 2021b)) is also a research hotspot in the computer vision community, some notable methods have been proposed to address both label shift and covariate shift, such as ViM (Wang et al., 2022), KL-Matching (Hendrycks et al., 2022) and Mahalanobis (Lee et al., 2018), but they have not been evaluated on malware domain. This is one of the reasons why this study subdivides concept drift. Our goal is to provide fair, comprehensive evaluation data for malware concept drift detection.

After enough drift samples have been detected and analyzed, the model needs to be retrained. This paper does not focus on specific retraining strategies (Chen et al., 2023). Instead, we are curious about how machine learning operates on packed malware samples in model retraining. Malware researchers have often assumed that packing would hinder machine learning to build effective classifiers on static malware features (Aghakhani et al., 2020). Surprisingly, recent classifiers always tell success stories, leading many experts to wonder whether the test benchmarks are biased (few datasets provide both clear family and packer tags). This is another reason why the covariate shift is subdivided into packed and evolved families in this work. We expect to also provide data for studying how static feature-based machine learning operates on packed families.

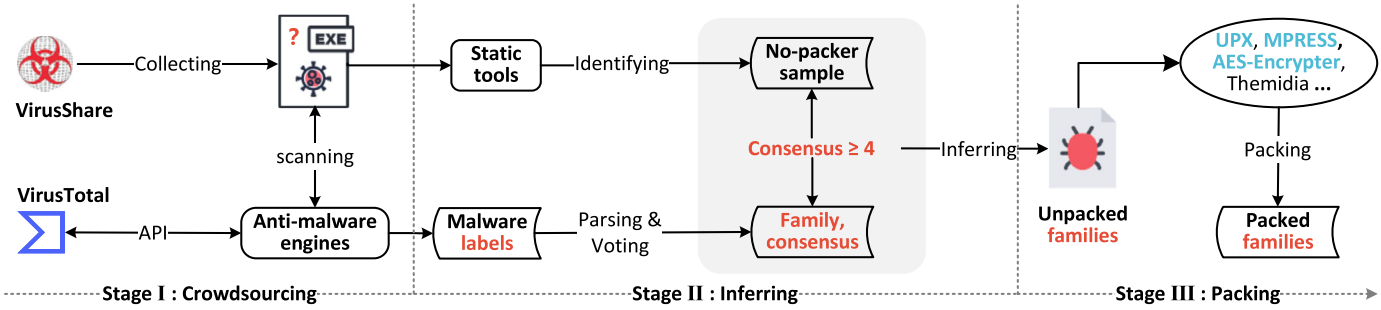



**Fig. 2.** The workflow for constructing BenchMFC from large-scale candidates. **Stage I:** collecting candidates and scanning with anti-malware engines to obtain malware labels. **Stage II:** combining static packer identification tools with malware family consensus to infer unpacked family samples. **Stage III:** creating packed families on demand by applying different packers.

### 2.3. Why a new benchmark dataset

To achieve the above goals, a dataset is needed, for which we can determine whether each sample is 1) packed or unpacked, 2) which family it belongs to, and 3) when it first appeared. Focusing on Windows PE, over the past decades, only a handful of malware datasets are publicly available for the security community, notable examples include MalImg (Nataraj et al., 2011), BIG-2015 (Ronen and Radu, 2018), EMBER (Anderson and Roth, 2018), and so on. As illustrated in Table 1, none of these datasets provide clear packer, family, and timestamp tags for each sample at the same time. This limits the ability of researchers to conduct a fair comparison of existing concept drift detectors, develop advanced strategies, and study how static feature-based machine learning operates on packed families when retraining a model. This is the motivation why we introduce BenchMFC.

However, building such a fine-grained dataset is challenging. First is packing detection, packing here means the original code of the executable sample exists in the file but is not in an executable form (it is encrypted, compressed, or transformed), and the original instructions are subsequently restored and executed during runtime. Latest Packware (Aghakhani et al., 2020) introduces a hybrid approach to detect packing, which combines the dynamic tool DPI (Ugarte-Pedrero et al., 2015) and several static tools. The DPI tool can precisely trace the unpacking behavior of runtime packers by monitoring *write-then-execute* behavior. A sample is regarded as unpacked if and only if the DPI tool does not detect the unpacking behavior and the static tools do not identify packers. However, this dynamic-based approach is time-consuming, it is not desirable for large-scale candidates. Second is family annotation, while several efforts have been made to address crowdsourcing malware annotation (Hurier et al., 2017; Sebasti'an and Caballero, 2020), it is still an open problem to infer truth from inconsistent anti-malware engines. As such, this paper introduces a novel crowdsourcing malware annotation pipeline (Section 3.2), which unifies packing detection and family annotation as a consensus inference problem to prevent costly packing detection.

## 3. Constructing BenchMFC dataset

In this section, we describe how to construct BenchMFC. Our goal is to provide a unified, well-curated, and comprehensive dataset, for which we can determine whether each sample is 1) packed or unpacked, 2) which family it belongs to, and 3) when it first appeared.

To achieve the goal, we first infer unpacked family samples with timestamps and then packed families can be created on demand by applying different packers. As illustrated in Fig. 2, the workflow includes three stages, **Stage I:** uploading samples to VirusTotal (VirusTotal, 2013) to collect malware labels from 70+ anti-malware engines for crowdsourcing malware annotation. **Stage II:** for each sample, extracting family tags from malware labels by the latest TagClass (Yongkang et al., 2023) and voting to obtain its *family consensus* among anti-malware engines. If there are $C_i$ engines that label the sample as family $k$, and $C_i$ is the highest vote, then the family consensus for the sample is noted as $C_i$. In addition, samples need to be analyzed by static packer identification tools PEiD (PEiD, 2019) and DIE (Horsicq, 2019). A sample is treated as an unpacked sample of family $k$ if and only if no packers are recognized and its family consensus $C_i \geq 4$. **Stage III:** creating packed families on demand by applying different packers. Next, we describe exactly how each stage is implemented.

### 3.1. Crowdsourcing malware annotation

The first task in constructing BenchMFC involves collecting candidate samples and corresponding malware labels. We collect 9.4 million candidates from ViusShare (Forensics, 2011) and upload these samples to VirusTotal (VirusTotal, 2013) to collect *malware labels* from 70+ anti-malware engines. To estimate the time of emergence of the sample, we take the minimum of the time the sample was collected and the time VirusTotal (VirusTotal, 2013) recorded it as the *first seen* of the sample. Finally, 21.2 million malware labels are collected. Malware labels include tags of classes: *type, platform, family,* and *modifier*, and are widely used to annotate malware families and provide evaluation benchmarks (Cozzi et al., 2018; Kotzias et al., 2019), which is referred to as crowdsourcing malware annotation.

Formally, consider a set of candidate samples $D = \{< x_i, y_i >\}_{i=1}^{I}$, where each sample $x_i$ has an unknown true family tag $y_i$ that belongs to a set of families $\{a_k\}_{k=1}^{K}$. These samples are uploaded to a set of anti-malware engines $\{e_j\}_{j=1}^{J}$ to collect malware labels $l_{ij}$. Each engine $e_j$ either does not response sample $x_i$ or just responses it only once. The research problem of crowdsourcing malware annotation is to estimate the golden truth $\hat{y}_i$ for each sample $x_i$ and minimize the empirical error, where the long-standing challenge lies in extracting family tags from massive malware labels and inferring unknown truth from inconsistent anti-malware engines.

### 3.2. Inferring unpacked family samples

Unlike previous work (Hurier et al., 2017; Sebasti'an and Caballero, 2020; Sebasti'an et al., 2016), BenchMFC aims to collect unpacked family samples with solid ground truth. As such, BenchMFC unifies packing detection and family annotation as a consensus inference problem to receive only strongly-tagged samples while preventing costly packing detection, called consensus-based unpacked family inferring (CUFI). Specifically, CUFI relies on the latest TagClass (Yongkang et al., 2023) to extract family tags and calculate family consensus, utilizes the *n-vote* scheme (Sebasti'an et al., 2016) to receive family tags of strongly-tagged samples, and combines static packer identification tools PEiD (PEiD, 2019) and DIE (Horsicq, 2019) to infer unpacked family samples. Let's go through the above process in turn.

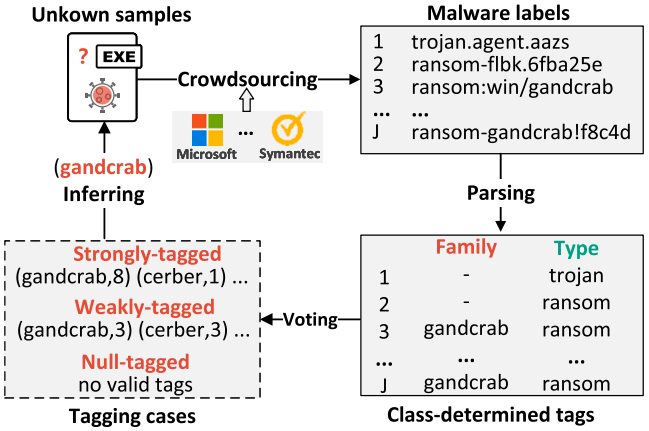



**Fig. 3.** Illustration of BenchMFC receiving strongly-tagged samples.

Consider a sample $x_i$ with parsed family tags $\boldsymbol{t}_i = \{t_{ij}\}_{j=1}^{J}$, $t_{ij} \in \{1, ..., K\}$. Family $k$ appears $N_k$ times, *i.e.*, $N_k^{(i)} = \sum_{j=1}^{J} \mathcal{I}(t_{ij} = k)$. The ***family consensus*** of sample $x_i$ is

$$C_i = \max\{N_k^{(i)}\}, \quad k \in [1, ..., K]. \tag{1}$$

As illustrated in Fig. 3, after paring, there may be samples of three tagging cases: 1) *null-tagged* samples, these samples receive no valid family tags, meaning that the ground truth of these samples cannot be inferred without introducing other knowledge; 2) *weakly-tagged* samples, the samples hold controversial tags reported by a close number of engines; and 3) *strongly-tagged* samples, these samples have clear and unambiguous tags, and their ground truth can be easily inferred.

Formally, sort $N_k^{(i)}$ in descending order from $N_{(1)}^{(i)}$, $N_{(2)}^{(i)}$ to $N_{(K)}^{(i)}$, strongly-tagged samples are defined as

$$x_i \in \text{strongly-tagged} \Longleftrightarrow N_{(1)}^{(i)} - N_{(2)}^{(i)} \geq 3. \tag{2}$$

To provide solid ground truth, the *n-vote* scheme is used to receive family tags of strongly-tagged samples, the validity and robustness of which have been demonstrated in previous studies (Zhu et al., 2020). For the *n-vote* scheme, $\hat{y}_i$ is determined by

$$\hat{y}_i = \begin{cases} \text{argmax}_k\{N_k^{(i)}\}, & \text{if } max(N_k^{(i)}) \geq n, \\ 0, & \text{if } max(N_k^{(i)}) < n. \end{cases} \tag{3}$$

Note that $\hat{y}_i = 0$ means *invalid voting*. The insight of BenchMFC is that if a sample is packed by common packers, it would be easy to identify by static tools, and if a sample is protected by uncommon packers, it will be more difficult for inconsistent anti-malware engines to reach a high family consensus on it. In other words, by combining static tools with family consensus and tuning the hyperparameter $n$, it would be possible to infer unpack family samples, called consensus-based unpacked family inferring (CUFI), which is defined as

$$\begin{aligned} &x_i \in \text{unpacked family samples} \Longleftrightarrow \\ &\text{no packer identified and } C_i \geq n. \end{aligned} \tag{4}$$

We measure this process using precision and recall, where precision indicates the proportion of true unpacked samples among the inferred unpacked samples and recall indicates how many true unpacked samples are inferred.

To evaluate the performance of CUFI, Packware (Aghakhani et al., 2020) is used to build a benchmark with 10,000 packed malicious samples and 10,000 unpacked malware family samples. Static tools PEiD (PEiD, 2019) and DIE (Horsicq, 2019) are used to build a baseline, if none of the tools identify a packer then the sample is considered to be unpacked, otherwise it is regarded as packed. The static baseline achieves 99.50% recall and 81.90% precision, signature-based packer identification approaches do miss a portion of packed samples. For

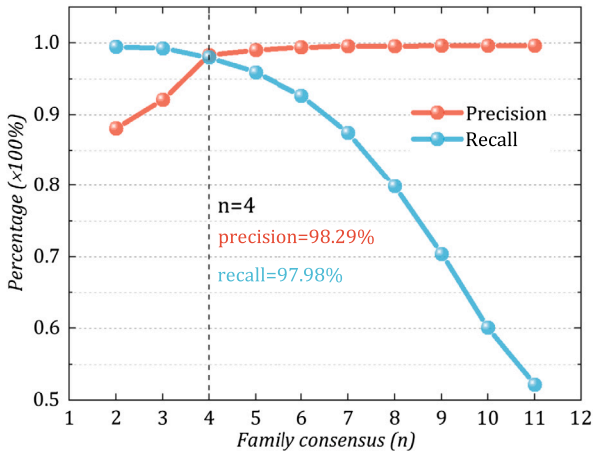



**Fig. 4.** Illustration of consensus-based unpacked family inferring (CUFI). *BenchMFC combines static tools and family consensus to infer unpacked family samples. For common packers, static tools can easily identify them. For uncommon packers, it is more difficult for inconsistent anti-malware engines to reach a high level of consensus.*

CUFI, as shown in Fig. 4, as the family consensus increases, the precision of the received unpacked family samples tends to increase and the recall tends to decrease. Specifically, when $n = 4$, CUFI achieves 98.29% precision and 97.98% recall in collecting unpacked family samples.

CUFI does not perform generic packing detection, it is a strategy to infer unpacked family samples from large-scale candidates to avoid expensive packing detection. Of the collected 9.4 million candidates, 7.2 million (76.6%) samples have family consensus $\geq 4$, 5.6 million (59.6%) samples are marked as strongly-tagged, and 4.7 million (50.0%) samples are identified as no packer by the static tools. According to CUFI, 4.1 million (43.6%) samples are inferred as unpacked samples of 3,049 families. We filter out small families with $\leq 50$ samples, and downsample large families to 1000 samples. Finally, BenchMFC includes 223 K unpacked samples of 526 malware families, of which about 25% of families have 1000 samples, 25% of families have 50-100 samples, and the remaining 50% of families have an average of 342 samples.

### 3.3. Creating packed families

BenchMFC provides scripts to create UPX, Petite, MPRESS and AES-Encrypter packed families. Similarly, researchers can easily make other packed families on their research needs. Malware researchers have often assumed that packing would hinder machine learning to build effective classifiers on static malware features (Aghakhani et al., 2020). However, recent studies always tell success stories, leading many experts to wonder whether the test benchmarks are biased (few datasets provide clear family and packer tags), or whether the classifiers just learn spurious correlations. This is one of the reasons why BenchMFC subdivides packed families from concept drift, it expects to provide data to study how static feature-based machine learning operates on packed families when retraining the classifiers.

Packing is widely used by malware authors to evade malware classifiers based on static features. Using the static packer identification tools PEiD (PEiD, 2019) and DIE (Horsicq, 2019), 4.5 million (48.9%) candidates are identified with packers. Considering that static tools only recognize known packers, the actual packed ratio should be higher. Fig. 5 lists the distribution of the identified packers. UPX is the most popular packer, followed by MPRESS, Petite, and so on. It's worth noting that AES-Encrypter (Aghakhani et al., 2020) is a strong and complete encryption packer, which encrypts a sample $p$ using AES with a random key, and injects the encrypted binary as an overlay of a new sample $\hat{p}$. The limitation of static feature-based machine learning for

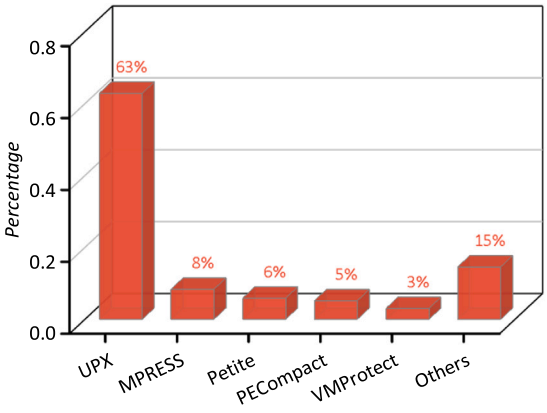



**Fig. 5.** The distribution of packers identified in the collected candidates. *4.5 million (48.9%) candidates are identified with packers. Packed families can be created on researchers' needs by applying different packers.*

malware detection has been discussed in recent work (Aghakhani et al., 2020), but an equally thorough study of malware family classification has yet been present. We believe that BenchMFC can offer unparalleled opportunities to researchers in this field.

## 4. Three important properties of BenchMFC

In this section, we outline the properties of BenchMFC after quality-controlled constructing, shedding light on its strengths in supporting research on three types of malware concept drift: 1) packed families, 2) evolved families, and 3) unseen families.

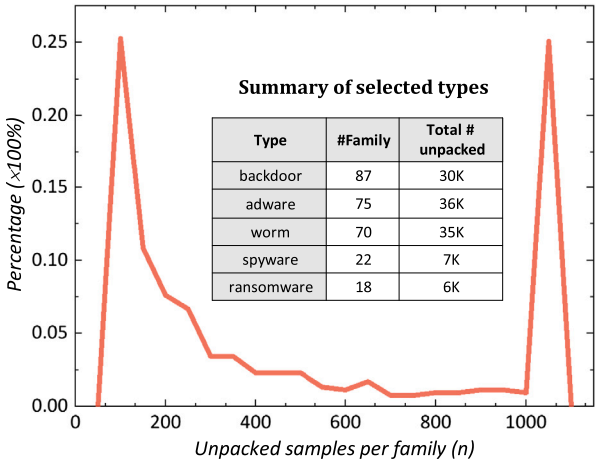



**Fig. 6.** Unpacked families of BenchMFC. ***Curve:*** *Histogram number of unpacked samples per family. About 25% of families have 1000 unpacked samples.* ***Table:*** *Summary of selected types.*

***Unpacked families.*** BenchMFC provides unpacked samples with solid family ground truth (Section 3.3), researchers can easily create corresponding packed families to simulate various concept drift scenarios caused by packing according to their research needs in the future. BenchMFC includes 223 K unpacked samples of 526 families. As illustrated in Fig. 6, over 25% of families have 1000 unpacked samples, about 25% of families have 50-100 unpacked samples, and the remaining 50% of families have an average of 342 unpacked samples. To the best of our knowledge, this is the first malware family dataset with clear and solid packer tags available to the security community, which will facilitate research on trustworthy malware classification based on static features.

***Evolving families.*** In support of research on evolved families, BenchMFC organizes samples by the time of their appearance. BenchMFC takes the minimum of the time the sample was collected and the time VirusTotal (VirusTotal, 2013) recorded the sample as an estimate of its *first seen* time. As shown in Fig. 7, over 80% families evolve ≥ 4 years, for example, *ranmnit* appears between 2012 and 2018, and *gandcrab* appears between 2018 and 2020. Benefit from this property, temporal analysis of different malware families can be easily conducted by exploiting their samples.

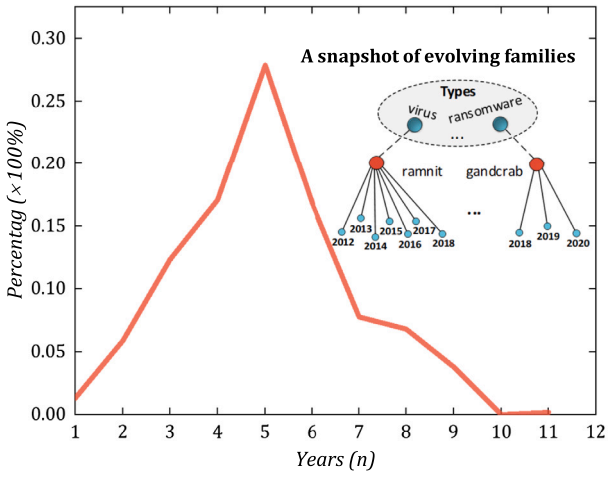



**Fig. 7.** Evolving families of BenchMFC. ***Curve:*** *Histogram of family evolving years, over 80% families evolve ≥ 4 years.*

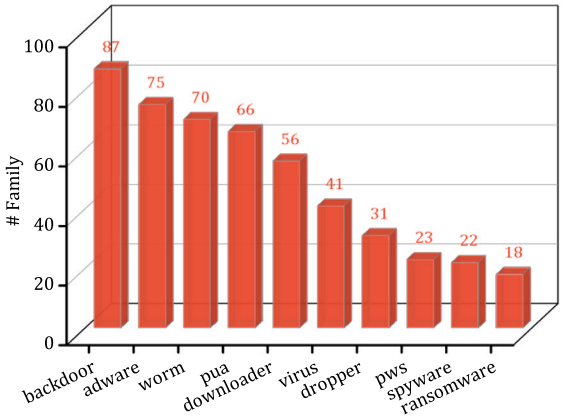



**Fig. 8.** Diverse families of BenchMFC. *BenchMFC contains 526 families of 25 common malware types, only top 10 types are listed.*

***Diverse families.*** BenchMFC provides diverse families to simulate label shift, *i.e.,* the appearance of "unseen families" during test phase that are not included in the training data. BenchMFC contains 526 families, covering 25 common malware types, making it one of the most diverse malware family datasets. As illustrated in Fig. 8, *backdoor, adware, worm* are the top three malware types, and the notorious *ransomware* contains 18 families. The diverse families of BenchMFC can support the study of concept drift introduced by label shift.

## 5. Experiments

In this section, we provide two case studies to illustrate the applications of BenchMFC in ***Q1: concept drift detection*** and ***Q2: model retraining***. The first set of experiments underlines the comprehensiveness of BenchMFC in supporting multi-type malware concept drift detection. The second set of experiments highlights the advantages of BenchMFC for studying how static feature-based machine learning operates on packed families when retraining a model.

**Table 2**
Experimental families sampled from BenchMFC. *G2 is to simulate unseen families in G1. P2 represents evolving samples that appeared after P1.*

| Type | Family | Group | #P1 | #P2 |
|---|---|---|---|---|
| backdoor | simda | G1 | 500 | 500 |
| | hupigon | G2 | 500 | - |
| ransomware | gandcrab | G1 | 500 | 500 |
| | wannacry | G2 | 500 | - |
| worm | yuner | G1 | 500 | 500 |
| | vobfus | G2 | 500 | - |
| adware | hotbar | G1 | 500 | 500 |
| | imali | G2 | 500 | - |
| pws | fareit | G1 | 500 | 500 |
| | onlinegames | G2 | 500 | - |
| spyware | zbot | G1 | 500 | 500 |
| | lydra | G2 | 500 | - |
| downloader | upatre | G1 | 500 | 500 |
| | zlob | G2 | 500 | - |
| virus | parite | G1 | 500 | 500 |
| | virut | G2 | 500 | - |

### 5.1. Setup

The following experiments will focus on concept drift of machine learning classifiers based on static malware features. Let's first introduce in turn the terms, static features, machine learning classifiers, experimental data, concept drift detectors, and evaluation metrics, that will be used later.

***Terms.*** Next, we will frequently use the terms "evolving", "packing" and "evolved", "packed". The "ing" form only represents the state of the sample, and the "ed" form represents the type of concept drift. For example, an evolving sample just means that the sample appears after the training data, while an evolved sample means that the sample appears after the training data and cannot be correctly recognized by the classifier.

***Static features.*** We consider two groups of typical static malware features: 1) **Byte** features (Raff et al., 2018a), the raw byte sequences extracted from executable samples, default is $2^{20}$ bytes, and 2) **EMBER** features (Anderson and Roth, 2018), a collection of nine sets of static features: byte histogram, byte entropy histogram, strings, general file info, header info, section info, imports info, exports info, and data directories info, forming a 2381 dimension of vector. More details about EMBER can be found in.[1]

***Machine learning classifiers.*** Using the above features, three common classifiers are trained:

- **GBDT** (Ke and Meng, 2017) trains decision trees to classify malware by gradient boosting algorithms. We train the GBDT model via EMBER features (Anderson and Roth, 2018), and utilize the same hyperparameters as the EMBER's implementation (Anderson and Roth, 2018).
- **MalConv** (Raff et al., 2018a) inputs raw bytes of samples and trains a CNN network to classify malware. Hyperparameters are set as described in the original paper.
- **MLP** (Hornik et al., 1989) is a fully connected feedforward neural network. We construct a three-layer MLP backbone to classify malware, each layer includes 1024, 512, and 256 hidden units. The EMBER features are used to train it.

***Experimental data.*** To conduct experiments, we sample 16 malware families from 8 malware types. As illustrated in Table 2, two families are selected for each malware type, marked as G1 and G2. G2 is used to simulate unseen families in G1. For each family, samples are divided into two periods, marked as P1 and P2. P2 denotes evolving samples that appeared after P1.

[1] https://github.com/elastic/ember.

**Table 3**
Summary of experimental data. *Element denotes the composition of the data. G1, G2 and P1, P2 are from Table 2.*

| Name | Train:val:test | Element | packed |
|---|---|---|---|
| MFC | 6:2:2 | G1-P1 | ✘ |
| MFC-Unseen | 6:2:2 | G2-P1 | ✘ |
| MFC-Evolving | 6:2:2 | G1-P2 | ✘ |
| MFC-Packing | 6:2:2 | G1-P1 | ✔ |

Then, we combine the sampled data to simulate three types of malware concept drift. As illustrated in Table 3, the name column reveals the type of concept drift relative to the underlying MFC data. The element column reveals the composition of the data, G1, G2 and P1, P2 are from Table 2. Note that MFC-Packing is packed by UPX, MPRESS and AES-Encrypter (1:1:1). For each group of named data, samples are divided into training, validation, and test set (6:2:2). Each classifier is trained for 50 epochs on the training set, we select the best weights on its validation set and calculate the metrics on the corresponding test set.

***Concept drift detectors.*** The following experiments will discuss 10 notable concept drift detectors, 8 from the image domain: MaxSoftmax (Hendrycks and Gimpel, 2016), ODIN (Liang et al., 2018), Energy (Liu et al., 2020), Entropy (Chan et al., 2021), KLMatching (Hendrycks et al., 2022), Mahalanobis (Lee et al., 2018), MaxLogit (Basart et al., 2022), and ViM (Wang et al., 2022); and 2 from the security domain: Cade (Yang et al., 2021d) and Transcend (Barbero et al., 2022).

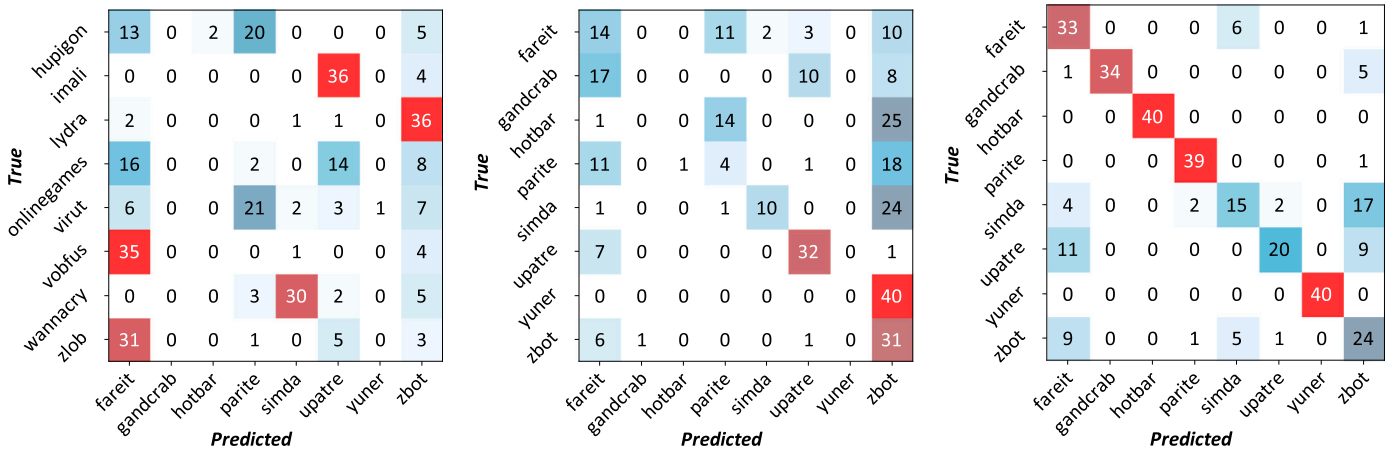
***Metrics.*** Similar to previous work (Aghakhani et al., 2020; Yang et al., 2021c), we use **accuracy** to evaluate the performance of the machine learning classifiers on the test set, higher scores mean better performance.

As for the performance of the concept drift detectors, the following two metrics are used: 1) **FPR95**, denoting the false positive rate (FPR) corresponding to a true positive rate (TPR) of 95%, lower scores indicate better performance; 2) **AUROC**, quantifying the area under the Receiver Operating Characteristic (ROC) curve, delineating the TPR-FPR relationship. The AUROC value serves as a probabilistic indicator denoting the likelihood of a positive concept drift sample yielding a higher detection score relative to a negative in-distribution sample.

### 5.2. Concept drift detection

In this group of experiments, we first train and test the GBDT, MalConv, and MLP classifiers on the MFC, MFC-Unseen, MFC-Evolving, and MFC-Packing data in turn, to figure out their performance on the experimental data without concept drift. Then we focus on the MLP classifier trained on MFC, and test it on the MFC-Unseen, MFC-Evolving, and MFC-Packing data to illustrate the impact of different types of malware concept drift. Finally, we evaluate the performance of notable concept drift detectors to detect the concept drift of the MLP classifier. The goal is to demonstrate that BenchMFC can serve as a central research resource for multi-type malware concept drift detection.

***Classifiers without concept drift.*** We train the classifiers on the training set of each experimental dataset (Table 3) and test them on the corresponding test set. The test and training sets of the same experimental dataset are independently and identically distributed (i.i.d.). As illustrated in Fig. 10, not surprisingly, all three classifiers achieve high accuracy ($\geq 94\%$) on the MFC, MFC-Unseen, and MFC-Evolving data, as all samples of which are unpacked, and there is no concept drift. Surprisingly, GBDT, MLP, and MalConv achieve 90.52%, 83.99%, and 84.92% accuracy on MFC-Packing, respectively. Recall that all samples of MFC-Packing are packed by AES-Encrypter, UPX and MPRESS




(a) Unseen families (b) Packing families (c) Evolving families

**Fig. 9.** Impact of different types of malware concept drift. *Confusion matrix of the MLP classifier trained on MFC and tested on MFC-Unseen, MFC-Packing and MFC-Evolving 3, the classifier achieves 96.25% accuracy on the MFC test set. (a) Most samples of an unseen family are misclassified into the same predefined families, misclassifications of most unseen families are not random. (b) The classifier achieves 29.84% accuracy, misclassifications of most packing families are random. (c) The classifier achieves 76.56% accuracy on the evolving families, samples of simda, upatre and zbot families have changed significantly.*

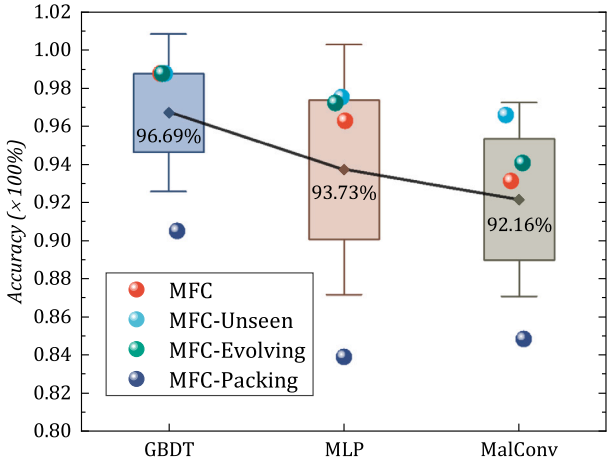



**Fig. 10.** Performance of the classifiers on the experimental data without concept drift. *Test and training sets are independently and identically distributed.*

(1:1:1), intuitively, the original static features of the families have been encrypted and compressed, the classifier should not achieve such good performance. This phenomenon leads us to the second case study that we will discuss later (Section 5.3). To sum up, without concept drift, all three classifiers achieve promising results on the four experimental datasets, and the average accuracy of each classifier are: 96.69% for GBDT, 93.73% for MLP, and 92.16% for MalConv.

***Impact of concept drift.*** Next, we show the impact of different types of malware concept drift. Without loss of generality, experiments in this part will use the test sets of MFC-Unseen, MFC-Evolving, and MFC-Packing to evaluate the MLP classifier trained on the MFC data.

A common case of malware concept drift is to introduce unseen families. We use the test set of MFC-Unseen to observe its impact. It is obvious that the MLP classifier trained on MFC cannot identify these unseen families, but we are curious whether the MLP classifier could show consistency in the misclassification. Fig. 9(a) illustrates the confusion matrix of the MLP classifier on the unseen families. Not surprisingly, it misidentifies all families. But most samples of an unseen family are misclassified into the same predefined families, misclassifications of the unseen *zlob, wannacry, vobus, lydra* and *imali* families are not random. That is, samples from the same unseen families can be identified by the classifier as having similar characteristics, but unfortunately it does not see these families during the training phase.

Another case is appearing packing families. To investigate the impact of it, we use the test set of MFC-Packing. We wonder if the classifier trained on the unpacked MFC families has some capabilities to recognize these packing families. As shown in Fig. 9(b), the MLP classifier achieves 29.84% accuracy. The confusion matrix indicates that misclassifications of most packing families are random, except *zbot* and *upatre*. Packing seems to pose a generalization challenge to the classifier, samples from the same packing families can no longer be identified by the classifier as having similar characteristics.

The last but rarely studied case is evolving families. To explore the impact of it, we use the test set of MFC-Evolving, where the samples all belong to the families in MFC, but appear later than the MFC samples. As illustrated in Fig. 9(c), the classifier obtains 76.56% accuracy and most samples of *zbot, upatre*, and *simda* are misclassified, indicating that samples of these families have changed significantly. Recall that the classifier achieves 96.25% accuracy on the MFC test set (Fig. 10), evolving families do make the performance of the classifier degrade. To summarize, the above experiments yield the following observation:

**Observation 1.** *Different types of malware concept drift have their own characteristics and can significantly degrade the performance of machine learning classifiers.*

***Detecting concept drift.*** To detect the above unseen, packed, and evolved samples, we evaluate nine notable concept drift detectors. Still focusing on the MLP classifier trained on the MFC training data, we randomly choose unseen, packed, and evolved samples from MFC-Unseen, MFC-Packing, and MFC-Evolving to form a new test set along with the test samples from MFC, where the ratio of concept drift (CD) samples to in-distribution (ID) samples is 1:1. The CD samples are regarded as positive, and the ID samples are regarded as negative. Metrics AUROC ↑ and FPR95 ↓ are used to evaluate the performance of the detectors. ↑ means the higher the better, and ↓ means the lower the better.

The core of a concept drift detector is a scoring function $\phi$ that maps an input feature $x$ to a scalar in $\mathbb{R}$, indicating the degree to which a sample is likely to be a drift sample. During the test phase, a specific threshold $\tau$ is determined to guarantee that the validation set maintains a given true-positive rate (TPR). The test sample is regarded as concept drift if $\phi(x) > \tau$ and in-distribution otherwise. The computer vision community has developed multiple scoring functions derived from three sources: 1) the probability, including the maximum softmax probabilities (Hendrycks and Gimpel, 2016) and the minimum KL-divergence between the softmax and the mean class-conditional dis-

**Table 4**
Concept drift detection for notable detectors. *The MLP classifier is trained on the MFC data, and unseen, packed and evolved samples are chosen from MFC-Unseen, MFC-Packing, and MFC-Evolving. The ratio of concept drift (positive) samples to in-distribution (negative) samples is 1:1. Both metrics AUROC and FRP95 are in percentage, ↑ means the higher the better, and ↓ means the lower the better. The best detector is emphasized in bold, and the 2nd and 3rd ones are underlined. The source field indicates the type of detector.*

| Classifier | Detector | Source | Unseen | | Packed | | Evolved | |
|---|---|---|---|---|---|---|---|---|
| | | | AUROC ↑ | FPR95 ↓ | AUROC ↑ | FPR95 ↓ | AUROC ↑ | FPR95 ↓ |
| MLP (EMBER) | MaxSoftmax (Hendrycks and Gimpel, 2016) | prob | 97.07 | 10.94 | 99.22 | 1.88 | 96.77 | 10.94 |
| | ODIN (Liang et al., 2018) | prob+grad | 97.08 | 10.63 | 99.23 | 1.88 | 96.75 | 10.94 |
| | Mahalanobis (Lee et al., 2018) | feat+label | **98.77** | 6.88 | 99.31 | 1.88 | **97.83** | 8.44 |
| | Energy (Liu et al., 2020) | logit | 97.68 | 7.81 | **99.49** | **1.25** | 97.17 | 9.38 |
| | Entropy (Chan et al., 2021) | logit | 97.19 | 10.31 | 99.36 | 1.88 | 96.85 | 10.94 |
| | MaxLogit (Basart et al., 2022) | logit | 97.65 | 7.81 | 99.44 | 1.25 | 97.14 | 9.38 |
| | KLMatching (Hendrycks et al., 2022) | prob | 86.71 | 61.87 | 93.61 | 49.06 | 85.37 | 49.06 |
| | Cade (Yang et al., 2021d) | feat | 94.08 | 14.54 | 95.92 | 10.06 | 92.18 | 24.87 |
| | ViM (Wang et al., 2022) | feat+logit | 97.55 | **5.00** | 97.92 | 4.06 | 97.19 | **5.94** |

tributions (Hendrycks et al., 2022); 2) the logit, including the maximum logit (Basart et al., 2022), Energy score (Liu et al., 2020) and Entropy score (Chan et al., 2021); and 3) the feature, including the minimum Mahalanobis distance between the feature and the class centroids (Lee et al., 2018) and virtual-logit matching (Wang et al., 2022). The security community also proposed a few methods, such as Transcend (Barbero et al., 2022) based on conformal prediction theory and Cade (Yang et al., 2021d) inspired by contrastive learning. It should be noted that we do not have access to Trancend's source code at this time, but Cade's experiments have shown that it outperforms Transcend. Therefore, please refer to our results on Cade for the performance of Transcend.

Table 4 illustrates the performance of detectors, both metrics AUROC and FRP95 are in percentage, the best detector is emphasized in bold, and the 2nd and 3rd ones are underlined. These results indicate that: 1) on the "unseen" and "evolved" data, Mahalanobis (Lee et al., 2018) obtains the largest AUROC while ViM (Wang et al., 2022) achieves the smallest FRP95; 2) on the "packed" data, Energy (Liu et al., 2020) reaches the best AUROC and FRP95; 3) most detectors have higher performance in detecting "packed" data than in detecting "unseen" and "evolved" data; and 4) ViM (Wang et al., 2022) is the only detector with relatively stable and well performance on all three types of concept drift.

Therefore, existing detectors have their own advantages in dealing with different types of concept drifts. For security users, the optimal detector needs to be selected based on the specific concept drift scenario. For researchers, the future research direction of concept drift detection should be to jointly optimize different types of concept drifts to develop advanced detectors that can better handle complex concept drift scenarios. The above results yield the following observation:

**Observation 2.** *Existing detectors have their own advantages in dealing different types of malware concept drift. Malware concept drift detection still has room for improvement.*

### 5.3. Model retraining

After detecting and analyzing enough drift samples, the model needs to be retrained. This paper does not focus on specific retraining strategies (Chen et al., 2023), we are curious about how machine learning operates on packed samples when retraining a model.

For static feature-based malware classifiers, retraining on unseen and evolved families does not introduce new generalization challenges, as samples still involve static features that represent their true selves. However, it is still an open problem whether these classifiers can relearn a decision boundary to distinguish packed families. Malware researchers have often assumed that static feature-based classifiers can not properly handle packed samples because in the most cases the static features of the original samples have been stripped away (Aghakhani et al., 2020). But our experiments on the MFC-Packing data (Fig. 10) show that this does not seem to be the case. In the following experiments, we will perform a preliminary investigation into the question: *does packing prevent malware family classification based on static features?* Further in-depth studies can be conducted in the future using clear packer and family tags of BenchMFC.

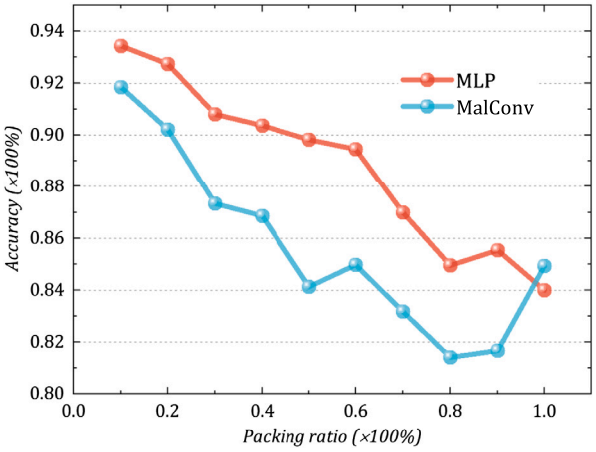



**Fig. 11.** Performance of the classifiers retraining at different packing ratios. *Samples are packed by UPX, MPRESS, and AES-Encrypter (1:1:1).*

***Different packing ratios.*** Recall that all samples of MFC-Packing are packed by UPX, MPRESS, and AES-Encrypter (1:1:1). We are curious if the rate of packing samples affects the performance of the classifier. As such, We adjust the packing ratio of samples in the MFC-Packing training, validation and test set from 10% to 100% with steps of 10%. Note that samples are still packed by UPX, MPRESS, and AES-Encrypter (1:1:1). For each packing ratio, we retrain the MLP and MalConv classifiers on the new training set and evaluate them on the corresponding test set. It is generally accepted that classifiers based on static features are not good at classifying packed families. However, the results in Fig. 11 indicate that, although the performance of all classifiers shows an overall decreasing trend as the packing ratio increases, MLP and MalConv perform relatively well on all packing ratios, with all accuracies $\geq 80\%$. In addition, the MLP classifier outperforms the MalConv classifier on all packing rates except 1.0.

***Strong encryption.*** The above results make us wonder how the classifiers perform on the strongly packed data. We choose single AES-Encrypter (Aghakhani et al., 2020) to pack samples, it encrypts a sample $p$ using AES with a random key, and injects the encrypted binary as an overlay of a new sample $\hat{p}$. The static features of the original sample are thus all encrypted and stripped. Intuitively, it should be difficult to retrain an effective static feature-based classifier on the AES-Encrypter packed samples to distinguish the original families.

To figure out this strong encryption case, we repeat the experiments shown in Fig. 11, but only AES-Encrypter is used to pack samples at this time. The results in Fig. 12 show that the MalConv classifier still per-

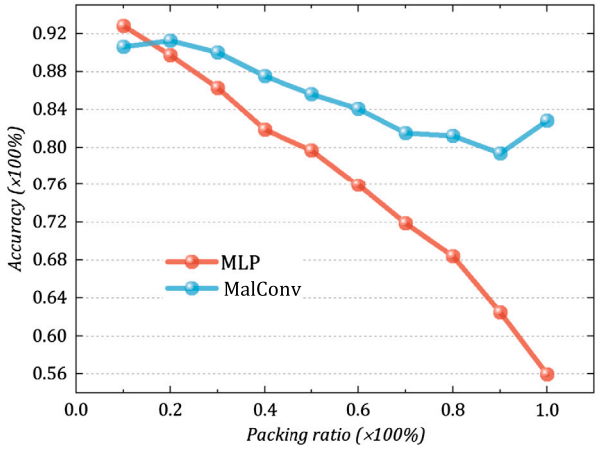



**Fig. 12.** Performance of the classifiers retraining on the strongly encrypted samples. *Samples are packed by AES-Encrypter.*

forms relatively well on all packing ratios, with accuracies $\geq 79\%$. But the performance of the MLP classifier degrades sharply with the increasing of packing ratio. Specifically, the accuracy of MLP is 79.69% when the packing ratio is 0.5 and drops to 55.94% when the packing ratio is 1.0. Recall that the MLP classifier is trained on the EMBER features (Anderson and Roth, 2018), which are some high-level static features about file structure, section information, and so on. When a sample is packed by AES-Encrypter, all this information of the original sample is encrypted. While the MalConv classifier is trained on the raw byte features (Raff et al., 2018a), when a sample is packed by AES-Encrypter, the byte sequences of the original sample become the encrypted byte sequences of a new sample. From the experimental results, these encrypted byte sequences contain richer signals of the original samples, making the MalConv classifier outperform the MLP classifier based on EMBER features. The above experiments lead to the following observation:

**Observation 3.** *Packers do preserve some kind of signals that appear to be "effective" for machine learning classifiers. However, the robustness of these signals requires further research.*

## 6. Conclusion

This paper presents a new benchmark dataset for malware family classification under concept drift, named BenchMFC. It complements the existing PE malware datasets while offering new capabilities. BenchMFC includes 223 K unpacked samples from 526 families that evolve years, it can support research on three types of malware concept drift: 1) unseen families, 2) packed families, and 3) evolved families. To infer unpacked family samples from large-scale candidates, BenchMFC introduces a novel crowdsourcing annotation pipeline, which unifies packing detection and family annotation as a consensus inference problem, preventing costly packing detection and outputting clean unpacked family samples. Moreover, we provide two case studies to illustrate the application of BenchMFC on concept drift detection and model retraining. The extensive experiments compare notable concept drift detectors and explore how static feature-based machine learning operates on packed families when retraining a model. We believe that BenchMFC can serve as a central research resource for trustworthy malware family classification under concept drift.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Acknowledgement

This work was supported by the National Nature Science Foundation of China under Grant 62372297, 62202303.

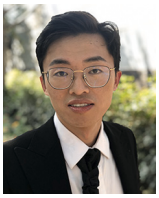
## References

Agarwal, S., Raj, G., 2018. Frame: framework for real time analysis of malware. In: International Conference on Cloud Computing. Data Science & Engineering, pp. 14–15.
Aghakhani, H., Gritti, F., Mecca, F., 2020. When malware is packin' heat; limits of machine learning classifiers based on static analysis features. In: Network and Distributed Systems Security (NDSS).
Anderson, H.S., Roth, P., 2018. Ember: an open dataset for training static pe malware machine learning models. ArXiv preprint arXiv:1804.04637.
Barbero, F., Pendlebury, F., Pierazzi, F., Cavallaro, L., 2022. Transcending transcend: revisiting malware classification in the presence of concept drift. In: IEEE Symposium on Security and Privacy (SP), pp. 805–823.
Basart, S., Mantas, M., Mohammadreza, M., Jacob, S., Dawn, S., 2022. Scaling out-of-distribution detection for real-world settings. In: International Conference on Machine Learning (ICML).
Bendale, A., Boult, T., 2015. Towards open world recognition. In: IEEE Conference on Computer Vision and Pattern Recognition, pp. 1893–1902.
Chan, R., Rottmann, M., Gottschalk, H., 2021. Entropy maximization and meta classification for out-of-distribution detection in semantic segmentation. In: International Conference on Computer Vision (ICCV), pp. 5128–5137.
Chen, Y., Ding, Z., Wagner, D., 2023. Continuous learning for Android malware detection. In: USENIX Security Symposium.
Cozzi, E., Graziano, M., Balzarotti, D., 2018. Understanding Linux malware. In: IEEE Symposium on Security and Privacy (SP), pp. 161–175.
Damodaran, A., Troia, F.D., Visaggio, C.A., Austin, T.H., Stamp, M., 2015. A comparison of static, dynamic, and hybrid analysis for malware detection. J. Comput. Virol. Hacking Tech. 13, 1–12.
Demetrio, L., Biggio, B., Lagorio, G., Roli, F., Armando, A., 2021. Functionality-preserving black-box optimization of adversarial windows malware. IEEE Trans. Inf. Forensics Secur. 16, 3469–3478.
Dib, M., Torabi, S., Bou-Harb, E., Bouguila, N., Assi, C., 2022. Evoliot: a self-supervised contrastive learning framework for detecting and characterizing evolving iot malware variants. In: Asia Conference on Computer and Communications Security (AsiaCCS), pp. 452–466.
Fleshman, W., Raff, E., Zak, R., McLean, M., Nicholas, C., 2018. Static malware detection & subterfuge: quantifying the robustness of machine learning and current anti-virus. In: International Conference on Malicious and Unwanted Software (MALWARE). IEEE, pp. 1–10.
Forensics, C., 2011. Virusshare: online malware repository project. https://virusshare.com.
Harang, R., Rudd, E.M., 2020. Sorel-20m: a large scale benchmark dataset for malicious PE detection. ArXiv preprint arXiv:2012.07634.
Hendrycks, D., Gimpel, K., 2016. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In: International Conference on Learning Representations (ICML).
Hendrycks, D., Basart, S., Mazeika, M., Zou, A., Kwon, J., Mostajabi, M., Steinhardt, J., Song, D., 2022. Scaling out-of-distribution detection for real-world settings. In: International Conference on Machine Learning (ICML), pp. 8759–8773.
Hornik, K., Stinchcombe, M., White, H., 1989. Multilayer feedforward networks are universal approximators. Neural Netw. 2 (5), 359–366.
Horsicq, 2019. Detect it easy. https://github.com/horsicq/Detect-It-Easy.
Hurier, M., Suarez-Tangil, G., Dash, S.K., Bissyandé, T.F., Le Traon, Y., Klein, J., Cavallaro, L., 2017. Euphony: harmonious unification of cacophonous anti-virus vendor labels for Android malware. In: IEEE International Conference on Mining Software Repositories, pp. 425–435.
Jordaney, R., Sharad, K., Dash, S., 2017. Transcend: detecting concept drift in malware classification models. In: USENIX Security Symposium, pp. 625–642.
Ke, G., Meng, Q., 2017. Lightgbm: a highly efficient gradient boosting decision tree. In: Conference on Neural Information Processing Systems (NIPS), vol. 30.
Kotzias, P., Bilge, L., Caballero, J., 2019. Mind your own business: a longitudinal study of threats and vulnerabilities in enterprises. In: Network and Distributed Systems Security (NDSS).
Lee, K., Lee, K., Lee, H., Shin, J., 2018. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In: Advances in Neural Information Processing Systems (NIPS), vol. 31.
Li, Q., Mi, J., Li, W., Wang, J., Cheng, M., 2021. Cnn-based malware variants detection method for internet of things. IEEE Int. Things J. 8 (23), 16946–16962.
Liang, S., Li, Y., Srikant, R., 2018. Enhancing the reliability of out-of-distribution image detection in neural networks. In: International Conference on Learning Representations (ICLR).

Lindorfer, M., Kolbitsch, C., Milani Comparetti, P., 2011. Detecting environment-sensitive malware. In: International Symposium on Research in Attacks, Intrusions and Defenses (RAID), pp. 338–357.
Liu, W., Wang, X., Owens, J., Li, Y., 2020. Energy-based out-of-distribution detection. Adv. Neural Inf. Process. Syst. 33, 21464–21475.
Moser, A., Kruegel, C., Kirda, E., 2007a. Limits of static analysis for malware detection. In: Annual Computer Security Applications Conference (ACSAC), pp. 421–430.
Moser, A., Kruegel, C., Kirda, E., 2007b. Limits of static analysis for malware detection. In: Annual Computer Security Applications Conference (ACSAC), pp. 421–430.
Nataraj, L., Karthikeyan, S., Jacob, G., Manjunath, B.S., 2011. Malware images: visualization and automatic classification. In: International Symposium on Visualization for Cyber Security, pp. 1–7.
Or-Meir, O., Nissim, N., Elovici, Y., Rokach, L., 2019. Dynamic malware analysis in the modern era—a state of the art survey. ACM Comput. Surv. 52 (5), 1–48.
Oz, H., Aris, A., Levi, A., Uluagac, A.S., 2022. A survey on ransomware: evolution, taxonomy, and defense solutions. ACM Comput. Surv. 54 (11s), 1–37.
Paul, S., Stamp, M., 2021. Word embedding techniques for malware evolution detection. In: Malware Analysis Using Artificial Intelligence and Deep Learning, pp. 321–343.
PEiD, 2019. Signature-based packer detector. https://www.aldeid.com/wiki/PEiD.
Qiang, W., Yang, L., Jin, H., 2022. Efficient and robust malware detection based on control flow traces using deep neural networks. Comput. Secur., 102871.
Qiu, J., Zhang, J., Luo, W., Pan, L., Nepal, S., Xiang, Y., 2020. A survey of Android malware detection with deep neural models. ACM Comput. Surv. 53 (6), 1–36.
Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., Nicholas, C.K., 2018a. Malware detection by eating a whole exe. In: Association for the Advancement of Artificial Intelligence (AAAI).
Raff, E., Zak, R., Cox, R., Sylvester, J., Yacci, P., Ward, R., Tracy, A., McLean, M., Nicholas, C., 2018b. An investigation of byte n-gram features for malware classification. J. Comput. Virol. Hacking Tech. 14, 1–20.
Raffetseder, T., Kruegel, C., Kirda, E., 2007. Detecting system emulators. In: Information Security, pp. 1–18.
Ronen, R., Radu, M., 2018. Microsoft malware classification challenge. ArXiv preprint arXiv:1802.10135.
Sebasti'an, M., Rivera, R., Kotzias, P., 2016. Avclass: a tool for massive malware labeling. In: Recent Advances in Intrusion Detection (RAID), pp. 230–253.
Sebasti'an, S., Caballero, J., 2020. Avclass2: massive malware tag extraction from av labels. In: Annual Computer Security Applications Conference (ACSAC), pp. 42–53.
Shafer, G., Vovk, V., 2008. A tutorial on conformal prediction. J. Mach. Learn. Res. 9 (3).
Tran, K., Di Troia, F., Stamp, M., 2022. Robustness of image-based malware analysis. In: Silicon Valley Cybersecurity Conference. Springer, pp. 3–21.
Tupadha, L.S., Stamp, M., 2022. Machine learning for malware evolution detection. In: Artificial Intelligence for Cybersecurity. Springer, pp. 183–213.
Ucci, D., Aniello, L., Baldoni, R., 2019. Survey of machine learning techniques for malware analysis. Comput. Secur. 81, 123–147.
Ugarte-Pedrero, X., Balzarotti, D., Santos, I., 2015. Sok: deep packer inspection: a longitudinal study of the complexity of run-time packers. In: IEEE Symposium on Security and Privacy (SP), pp. 659–673.
VirusTotal, 2013. Online virus, malware and url scanner. https://www.virustotal.com.
Wadkar, M., Di Troia, F., Stamp, M., 2020. Detecting malware evolution using support vector machines. Expert Syst. Appl. 143, 113022.
Wang, H., Li, Z., Feng, L., Zhang, W., 2022. Vim: out-of-distribution with virtual-logit matching. In: Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4921–4930.
Yan, J., Yan, G., Jin, D., 2019. Classifying malware represented as control flow graphs using deep graph convolutional neural network. In: IEEE International Conference on Dependable Systems and Networks (DSN), pp. 52–63.
Yang, J., Wang, H., Feng, L., Yan, X., Zheng, H., Zhang, W., Liu, Z., 2021a. Semantically coherent out-of-distribution detection. In: IEEE/CVF International Conference on Computer Vision (ICCV), pp. 8301–8309.
Yang, J., Zhou, K., Li, Y., Liu, Z., 2021b. Generalized out-of-distribution detection: a survey. ArXiv preprint arXiv:2110.11334.
Yang, J., Wang, P., Zou, D., Zhou, Z., Ding, K., Peng, W., Wang, H., Chen, G., Li, B., Sun, Y., et al., 2022. Openood: benchmarking generalized out-of-distribution detection. In: Conference on Neural Information Processing Systems (NIPS), vol. 35, pp. 32598–32611.
Yang, J., Zhou, K., Liu, Z., 2023. Full-spectrum out-of-distribution detection. Int. J. Comput. Vis., 1–16.
Yang, L., Ciptadi, A., Laziuk, I., 2021c. Bodmas: an open dataset for learning based temporal analysis of pe malware. In: IEEE Symposium on Security and Privacy Workshop, pp. 78–84.
Yang, L., Guo, W., Hao, Q., 2021d. Cade: detecting and explaining concept drift samples for security applications. In: USENIX Security Symposium, pp. 2327–2344.
Yongkang, J., Gaolei, L., Shenghong, L., 2023. Tagclass: a tool for extracting class-determined tags from massive malware labels via incremental parsing. In: IEEE International Conference on Dependable Systems and Networks (DSN).
Zhang, X., Zhang, Y., Zhong, M., Ding, D., Cao, Y., Zhang, Y., Zhang, M., Yang, M., 2020. Enhancing state-of-the-art classifiers with api semantics to detect evolved Android malware. In: Conference on Computer and Communications Security (CCS), pp. 757–770.
Zhu, S., Shi, J., Yang, L., 2020. Measuring and modeling the label dynamics of online anti-malware engines. In: USENIX Security, pp. 2361–2378.



**Yongkang Jiang** received the bachelor degree in Information Science and Engineering from Southeast University and the master degree in Computer Science and Technology from Shanghai Jiao Tong University, Shanghai, China, in 2019. He is currently pursuing the PhD degree in Cyber Security at Shanghai Jiao Tong University. He has published several high-quality papers on international conferences including IEEE DSN, ICONIP, etc. His research interests include machine learning and malware analysis.



**Gaolei Li** received the bachelor degree in Electronic Information Engineering from Sichuan University and PhD degree in Cyber Security from Shanghai Jiao Tong University, Shanghai, China, in 2020. From Oct. 2018 to Sep. 2019, he visited the Muroran Institution of Technology, Muroran, Japan, supported by the China Scholarship Council Program. Now, he is an Assistant Professor in School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China. He has published more than 50 papers including IEEE TIFS, TDSC, AAAI, etc. His research interests include machine learning security, privacy-preserving, etc. He received best conference paper awards from the IEEE ComSoc CISM Committee, Chinese Association for Cryptologic Research (CACR) and student travel grant award for IEEE Globecom. He serves as a TPC member of AAAI 2023, ACM MM 2023, IEEE ICC 2018-2021, IEEE ISPA 2020-2021, ScalCom 2020&2021 and a reviewer of IEEE TIFS, IEEE TDSC, IEEE IoT-J, and IEEE TCCN.



**Shenghong Li** received the B.S. and M.S. degrees in electrical engineering from the Jilin University of Technology, Changchun, China, in 1993 and 1996, respectively, and the Ph.D. degree in radio engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 1999. Since 1999, he has been with Shanghai Jiao Tong University, Shanghai, China, as a Research Fellow, an Associate Professor, and a Professor, successively. In 2010, he was as a Visiting Scholar with Nanyang Technological University, Singapore. He has published over 80 papers, coauthored four books, and held ten granted patents. His current research interests include information security, signal and information processing, and artificial intelligence. He was a recipient of the First Prize of Shanghai Science and Technology Progress in China in 2013. He was elected for New Century Talent of Chinese Education Ministry and Shanghai dawn scholar in 2006 and 2007.



**Ying Guo** received the B.S. degree from the University of Electronic Science and Technology of China, in 2014, and the Ph.D. degree from Shanghai Jiao Tong University, in 2019. She is a Lecturer with the School of Computer Science and Technology, in North China University of Technology. Her research interests include machine learning and their applications, social network analysis, Artificial Intelligence security.